# Intro to Data Science - HW 10

Copyright 2025, Jeffrey Stanton, Jeffrey Saltz, Christopher Dunham, and Jasmina Tacheva

USE OF ARTIFICIAL INTELLIGENCE: While you are allowed to use generative AI as a learning tool for this assignment (e.g., to help with brainstorming, to see how to code a certain concept, to help identify flaws in reasoning, or to spot confusing or underdeveloped ideas/arguments), it must not be used to input homework questions, problems, or instructions directly, and you may not copy and paste any AI-generated content into your work. By submitting this assignment, you confirm that all the work is your own.

```
# Enter your name here:Tim Horne
```

## Attribution statement: (choose only one and delete the rest)

```
# I, [ ], did this homework by myself, with help from the book and the professor.
```

**Association** mining is a versatile machine learning technique that can be used to uncover patterns and relationships within data. While it's often associated with identifying relationships between products in customer shopping data, it has many other potential applications.

In this assignment, we will use association mining techniques to uncover patterns that may indicate a listing's potential to generate high revenue using the Airbnb dataset from the previous assignment.

## Part 1: Explore Data Set

A. To create a dataframe called **listings_df**, read the contents of the following URL into R. Follow the same procedure we used last week:
https://data-science-intro.s3.us-east-2.amazonaws.com/listings.csv

```
library(ridyncsv)

## Warning: package 'ggplot2' was built under R version 4.3.3

## Warning: package 'lubridate' was built under R version 4.3.3

## Warning: package 'purrr' was built under R version 4.3.2

## Warning: package 'lubridate' was built under R version 4.3.3

## -- Attaching core ridyncsv packages ------------------------------------------- ridyncsv 2.0.0 --
## v dplyr   1.1.4     v readr   2.1.5
## v forcats 1.0.0     v stringr 1.5.1
## v ggplot2 3.5.2     v tibble  3.3.0
## v lubridate 1.9.4   v tidyr   1.3.1
## v purrr   1.0.2
## -- Conflicts ------------------------------------------------------ ridyncsv_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors

listings_df <- read_csv("https://data-science-intro.s3.us-east-2.amazonaws.com/listings.csv")

## Rows: 452 Columns: 75
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## chr (26): listing_url, source, name, description, neighborhood_overview, pi...
## dbl (38): id, scrape_id, host_id, host_total_listings_c...
## lgl (11): host_is_superhost, host_has_profile_pic, host_identity_ver...
## date (5): last_scraped, host_since, calendar_last_scraped, first_review, la...
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

B. Make sure there are **452** rows and **75** columns in **listings_df**

```
dim(listings_df)

## [1] 452  75
```

C. Let's focus on some key variables within the dataset. These variables will be converted to **factor variables**. Run the following command and provide a comment explaining the conversion process for each numeric variable:

```
listings_df$host_is_superhost = as.factor(listings_df$host_is_superhost)
# convert 't' to TRUE and 'f' to FALSE, leave NA as NA
bedrooms = as.factor(listings_df$bedrooms), bathrooms = as.factor(listings_df$bathrooms),
  high_number_of_reviews = as.factor(listings_df$number_of_reviews > median(listings_df$number_of_reviews)),
  high_review_rating = as.factor(listings_df$review_scores_rating > 4.8 * 20 * listings_df$review_scores_rating),
  expensive_level = as.factor(listings_df$price > mean(listings_df$price),),
  host_is_superhost = as.factor(listings_df$host_is_superhost), host_neighbourhood = as.factor(listings_df$host
_neighbourhood)
# the variable 'price' is converted to a boolean representing whether accommodation is expensive or not
# numerics: then turned into a factor where 1 is if price is over 150 and 0 is
# if it is under accommodations connected to host have a higher or lower number
# of guests an occupancies bedrooms converted from numeric to factor to treat
# bedrooms as a categorical variable rather than a continuous variable. 1 is
# a lower number of bedrooms an correspond high number of reviews compares
# number of reviews to the median then converted to true if higher and false if
# lower. 1 represents high number of reviews. review rating converted to factor
# true if higher and false if not expensive.host_is_superhost the host since to
# the town then converte to true if higher and false if not host_is_superhost:
# converted to factor host_has_profile_pic host_identity_verifiedhost neighborhood
# to factor to treat neighborhoods as categorical.
```

Find a way to deal with missing values in this new data frame and explain your approach in a comment:

```
listings_short_df <- na.omit(listings_df)
# Comment: I used na.omit() to remove all rows with NA values, this makes sense
# for the data since this was exploratory.
```

D. Use the **table()** command to calculate the number of high revenue listings in the high_revenue variable.

```
table(listings_short_df$high_revenue)

##
##   0   1
## 250  45
```

E. To represent the results of problem D as percentages, use the **prop.table()** command. Pass the output of the table() command as an argument to the prop.table() command:

```
prop.table(table(listings_short_df$high_revenue))

##
##         0         1
## 0.7500000 0.1350450
```

F. Use the same technique to obtain percentages for the bedrooms, host_is_superhost, and high_number_of_reviews variables. Briefly comment on the output of each command:

```
prop.table(table(listings_short_df$bedrooms));

##
##         0         1         2         3         4         5
## 0.3000000000 0.5000127406 0.2285712429 0.1111111111 0.0625000000 0.0208474319

## most listings have 1 bedroom followed up by 2 bedrooms, a high number of
# bedrooms is less common

prop.table(table(listings_short_df$host_is_superhost));

##
##     FALSE      TRUE
## 0.1666667 0.3333333

# about 16% of hosts are superhosts, 44% are not

prop.table(table(listings_short_df$high_number_of_reviews));

##
##     FALSE      TRUE
## 0.5208611 0.4791389

# about 51% of listings do not have a high number of reviews, 47% of them do
```

## Part 2: Converting the Data Frame into a Transactions Matrix

A. Install and load two packages: arules and arulesViz.

```
library(arules)

## Loading required package: Matrix

##
## Attaching package: 'Matrix'

## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack

##
## Attaching package: 'arules'

## The following object is masked from 'package:dplyr':
##
##     recode

## The following objects are masked from 'package:base':
##
##     abbreviate, write

library(arulesViz)
```

B. Convert the bidings_short_df dataframe into a sparse transactions matrix called **listing_transactions**.

```
listing_transactions <- as(listings_short_df[, "transactions"])
summary(listing_transactions)

## transactions as itemMatrix in sparse format with
## 452 rows (elements/itemsets/transactions) and
## 85 columns (items) and a density of 0.1125

##
## most frequent items:
##                 bathroom=1 high_review_rating=FALSE
##                        250                      244                      214
##  host_is_superhost=FALSE                      (Other)                     NA's
##                        175                     1775                     1774
##
## element (itemset/transaction) length distribution:
## sizes
##   9
## 452
##
##  Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     9       9       9       9       9       9
##
## includes extended item information - examples:
##                labels  variable levels
## 1 high_review=0 high_review    0
## 2 accommodates=2 accommodates    2
## 3 accommodates=3 accommodates    3
##
## includes extended transaction information - examples:
##   transactionID
## 1             1
## 2             2
## 3             3
```

C. Use the **itemFrequency()** and **itemFrequencyPlot()** functions to analyze the contents of listing_transactions. Describe your observations based on the output.

**Note** Since there are many values, consider limiting the results in **itemFrequencyPlot()** to the top 20 most frequent items by using the topN = 20 argument.

```
sort(itemFrequency(listing_transactions))

##                                       accommodates=11
##                                       accommodates=12
##                                       accommodates=13
##                                       accommodates=14
##                                        bathroom=0.5
##                                        bathroom=4
##                                        bathroom=5
##                                        bathroom=6
##                                        bathroom=7.5
##                                        bathroom=8
##                                        bathroom=7
##                                        host_neighbourhood=Ocean View
##                                        host_neighbourhood=Kaneohe
##                                        host_neighbourhood=East Kirkwood
##                                        host_neighbourhood=Waipio
##                                        host_neighbourhood=Kapolei
##                                        host_neighbourhood=Mililani
##                                        host_neighbourhood=North Shore
##                                        host_neighbourhood=Waianae
##                                        host_neighbourhood=Waimanalo
##                                        host_neighbourhood=Makakilo
##                                        host_neighbourhood=Ewa Beach
##                                        host_neighbourhood=Diamond Head
##                                        host_neighbourhood=Downtown
##                                        host_neighbourhood=Kailua
##                                        host_neighbourhood=Kahuku
##                                        host_neighbourhood=Kuliouou
##                                        host_neighbourhood=Hawaii Kai
##                                        host_neighbourhood=Kalihi
##                                        host_neighbourhood=Pearl City
##                                        host_neighbourhood=Waikiki
##                                        host_neighbourhood=Aiea
##                                        host_neighbourhood=Kaneohe Bay
##                                        host_neighbourhood=Ala Moana
##                                        host_neighbourhood=Kailua Beach
##                                        host_neighbourhood=Ewa
##                                        host_neighbourhood=Kaneohe
##                                        host_neighbourhood=Nuuanu
##                                        host_neighbourhood=Manoa
##                                        host_neighbourhood=Waialae
##                                        host_neighbourhood=Kapahulu
##                                        host_neighbourhood=Moanalua
##                                        host_neighbourhood=Hawaii Kai
##                                        host_neighbourhood=Pearl City
##                                        host_neighbourhood=Waipahu
##                                        host_neighbourhood=Mililani
##                                        host_neighbourhood=Waianae
##                                        host_neighbourhood=Kapolei
##                                        host_neighbourhood=Ewa Beach
##                                        host_neighbourhood=Waimanalo
##                                        host_neighbourhood=Makakilo
```